Concepts

Logging

The platform provides logging for both backtesting and live trading using a high-performance logging subsystem implemented in Rust with a standardized facade from the `log` crate.

The core logger operates in a separate thread and uses a multi-producer single-consumer (MPSC) channel to receive log messages. This design ensures that the main thread remains performant, avoiding potential bottlenecks caused by log string formatting or file I/O operations.

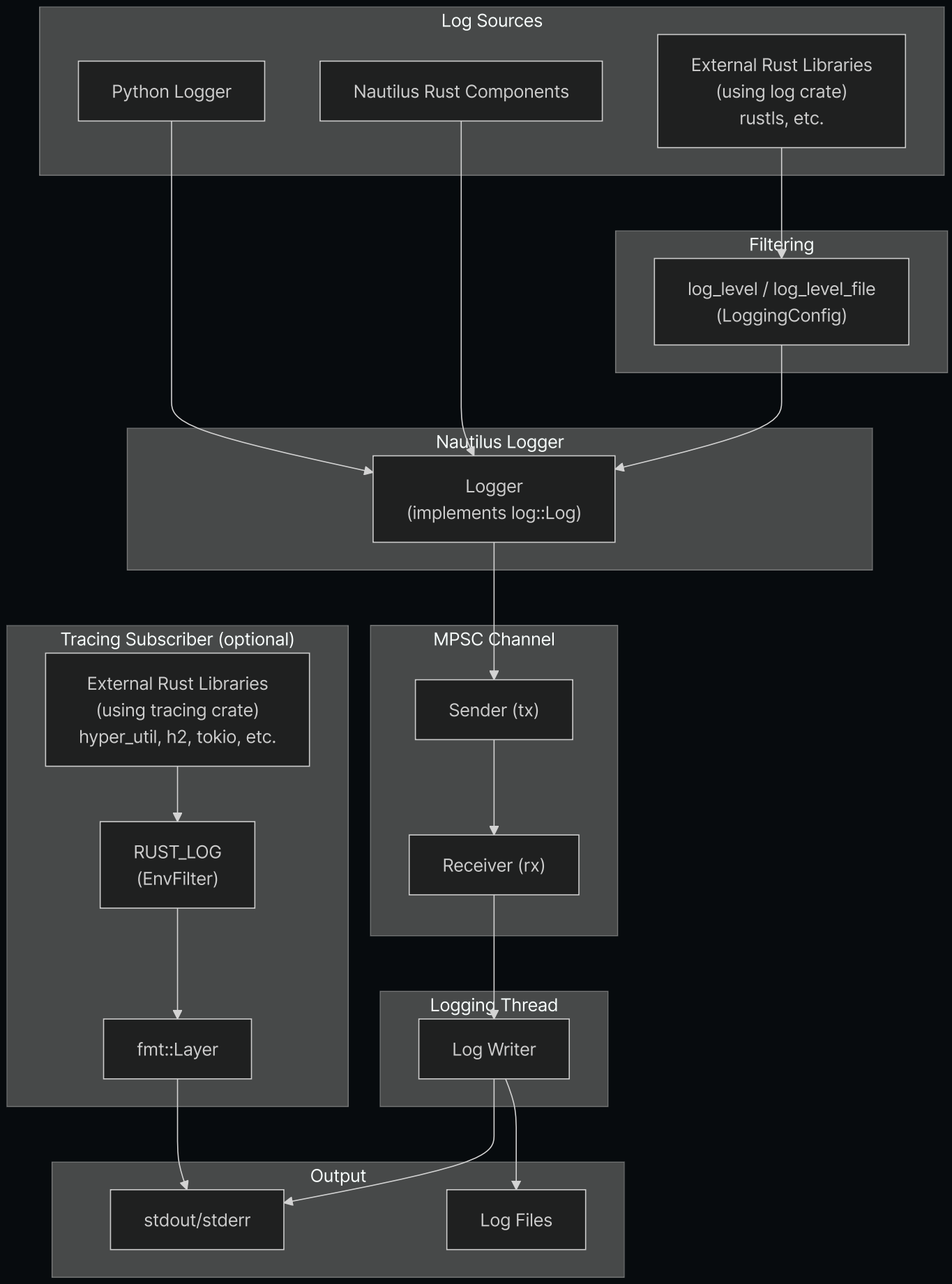
Logging output is configurable and supports:

- **stdout/stderr writer** for console output
- **file writer** for persistent storage of logs

> Infrastructure such as Vector can be integrated to collect and aggregate events within your system.

Architecture

The logging subsystem captures events from multiple sources and routes them through an MPSC channel to a dedicated logging thread:



- **Python and Nautilus components**: Log directly through the Nautilus Logger.
- **External `log` crate users**: Filtered by `log_level` / `log_level_file` in `LoggingConfig`.

- **External `tracing` crate users**: When enabled, output goes directly to stdout (separate from Nautilus logging), filtered by the `RUST_LOG` environment variable.
- **Logging thread**: All Nautilus log events are sent through an MPSC channel to a dedicated thread, ensuring the main thread isn't blocked by I/O operations.

Configuration

Logging can be configured by importing the `LoggingConfig` object. By default, log events with an 'INFO' `LogLevel` and higher are written to stdout/stderr.

Log level (`LogLevel`) values include the following (matching standard log level conventions).

The following log levels are supported:

- `OFF` - Disable logging.
- `TRACE` - Most verbose; only emitted by Rust components (cannot be generated from Python).
- `DEBUG` - Detailed diagnostic information.
- `INFO` - General operational messages.
- `WARNING` - Potential issues that don't prevent operation.
- `ERROR` - Errors that may affect functionality.

> You can set `TRACE` as a filter level to capture trace logs from Rust components, even though Python code cannot emit them directly.

See the `LoggingConfig` API Reference for further details.

Logging can be configured in the following ways:

- Minimum `LogLevel` for stdout/stderr.
- Minimum `LogLevel` for log files.
- Maximum size before rotating a log file.
- Maximum number of backup log files to maintain when rotating.
- Automatic log file naming with date or timestamp components, or custom log file name.

- Directory for writing log files.
- Plain text or JSON log file formatting.
- Filtering of individual components by log level.
- ANSI colors in log lines.
- Bypass logging entirely.
- Print Rust config to stdout at initialization.
- Optionally initialize logging via the PyO3 bridge (`use_pyo3`) to capture log events emitted by Rust components.
- Truncate existing log file on startup if it already exists (`clear_log_file`)

Standard output logging

Log messages are written to the console via stdout/stderr writers. The minimum log level can be configured using the `log_level` parameter.

File logging

Log files are written to the current working directory by default. The naming convention and rotation behavior are configurable and follow specific patterns based on your settings.

You can specify a custom log directory using `log_directory` and/or a custom file basename using `log_file_name`.

Log file formats:

- `None` (default) - Plain text format with `.log` extension.
- `"json"` - JSON format with `.json` extension, useful for log aggregation tools.

For detailed information about log file naming conventions and rotation behavior, see the Log file rotation and Log file naming convention sections below.

Log file rotation

Rotation behavior depends on both the presence of a size limit and whether a custom file name is provided:

- Size-based rotation:

- Enabled by specifying the `log_file_max_size` parameter (e.g., `100_000_000` for 100 MB).
- When writing a log entry would make the current file exceed this size, the file is closed and a new one is created.

- Date-based rotation (default naming only):
  - Applies when no `log_file_max_size` is specified and no custom `log_file_name` is provided.
  - At each UTC date change (midnight), the current log file is closed and a new one is started, creating one file per UTC day.
- No rotation:
  - When a custom `log_file_name` is provided without a `log_file_max_size`, logs continue to append to the same file.
  - Note: Size-based rotation takes precedence - if both a custom name and size limit are provided, rotation still occurs.
- Backup file management:
  - Controlled by the `log_file_max_backup_count` parameter (default: 5), limiting the total number of rotated files kept.
  - When this limit is exceeded, the oldest backup files are automatically removed.

Log file naming convention

The default naming convention ensures log files are uniquely identifiable and timestamped. The format depends on whether file rotation is enabled:

With file rotation enabled:

- Format: `{trader_id}_{%Y-%m-%d_%H%M%S:%3f}_{instance_id}.{log|json}`
- Example: `TESTER-001_2025-04-09_210721:521_d7dc12c8-7008-4042-8ac4-017c3db0fc38.log`
- Components:
  - `{trader_id}`: The trader identifier (e.g., `TESTER-001`).
  - `{%Y-%m-%d_%H%M%S:%3f}`: Full ISO 8601-compliant datetime with millisecond resolution.
  - `{instance_id}`: A unique instance identifier.
  - `{log|json}`: File suffix based on format setting.

Without size-based rotation (default naming):

- Format: `{trader_id}_{%Y-%m-%d}_{instance_id}.{log|json}`
- Example: `TESTER-001_2025-04-09_d7dc12c8-7008-4042-8ac4-017c3db0fc38.log`
- Components:
  - `{trader_id}`: The trader identifier.
  - `{%Y-%m-%d}`: Date only (YYYY-MM-DD).
  - `{instance_id}`: A unique instance identifier.
  - `{log|json}`: File suffix based on format setting.
- Note: With default naming and no size limit, logs rotate daily at UTC midnight.

Custom naming:

If `log_file_name` is set (e.g., `my_custom_log`):

- With rotation disabled: The file will be named exactly as provided (e.g., `my_custom_log.log`).
- With rotation enabled: The file will include the custom name and timestamp (e.g., `my_custom_log_2025-04-09_210721:521.log`).

Component log filtering

The `log_component_levels` parameter can be used to set log levels for each component individually. The input value should be a dictionary of component ID strings to log level strings: `dict[str, str]`.

Below is an example of a trading node logging configuration that includes some of the options mentioned above:

```
from nautilus_trader.config import LoggingConfig
from nautilus_trader.config import TradingNodeConfig

config_node = TradingNodeConfig(
    trader_id="TESTER-001",
    logging=LoggingConfig(
        log_level="INFO",
        log_level_file="DEBUG",
        log_file_format="json",
        log_component_levels={ "Portfolio": "INFO" },
    ),
    ... # Omitted
)
```

For backtesting, the `BacktestEngineConfig` class can be used instead of `TradingNodeConfig`, as the same options are available.

Environment variable configuration

The `NAUTILUS_LOG` environment variable provides an alternative way to configure logging using a semicolon-separated spec string. This is useful for Rust-only binaries or when you want to override logging settings without modifying code.

```
export NAUTILUS_LOG="stdout=Info;fileout=Debug;RiskEngine=Error;is_colored"
```

Supported keys:

Key	Type	Description
`stdout`	Log level	Maximum level for stdout output.
`fileout`	Log level	Maximum level for file output.
`is_colored`	Flag	Enable ANSI colors (default: true).
`print_config`	Flag	Print config to stdout at startup.
`log_components_only`	Flag	Only log components with explicit filters.
`<Component>`	Log level	Component-specific level (exact match).
`<module::path>`	Log level	Module-specific level (prefix match, Rust only).

Flags are enabled by their presence in the spec string (no value needed). Log levels are case-insensitive: `Off`, `Trace`, `Debug`, `Info`, `Warning` (or `Warn`), `Error`.

> For Rust-only binaries, setting `NAUTILUS_LOG` enables lazy initialization of the logging subsystem on first use, without requiring explicit `init_logging()` calls.

Components-only logging

When focusing on a subset of noisy systems, enable `log_components_only` to log messages only from components explicitly listed in `log_component_levels`. All other components are

suppressed regardless of the global `log_level` or file level.

Example (Python configuration):

```
logging = LoggingConfig(
    log_level="INFO",
    log_component_levels={
        "RiskEngine": "DEBUG",
        "Portfolio": "INFO",
    },
    log_components_only=True,
)
```

If configuring via the environment using the Rust spec string, include `log_components_only` alongside component filters, for example:

```
export NAUTILUS_LOG="stdout=Info;log_components_only;RiskEngine=Debug;Portfolio=Info"
```

Module path filtering (Rust only)

When using the `NAUTILUS_LOG` environment variable, you can filter by Rust module paths in addition to component names. Keys containing `::` are treated as module path filters with prefix matching, while keys without `::` are component filters with exact matching.

```
# Filter all adapters to Warn, but allow Debug for OKX specifically
export NAUTILUS_LOG="stdout=Info;nautilus_okx=Warn;nautilus_okx::websocket=Debug"
```

The longest matching prefix takes precedence. In the example above, `nautilus_okx::websocket::handler` would use the `Debug` level (longer prefix), while `nautilus_okx::data` would use `Warn`.

Rust log macros automatically capture the module path when no explicit component is provided. This enables module-level filtering to work with standard logging calls.

Module path filtering is only available via the `NAUTILUS_LOG` environment variable. The Python `log_component_levels` configuration uses component name matching only.

> ⚠ If `log_components_only=True` (or `log_components_only` is present in the spec string) and `log_component_levels` is empty, no log messages will be emitted to stdout/stderr or files. Add at least one component filter or disable components-only logging.

Log colors

ANSI color codes improve log readability in terminals. In environments that do not support ANSI color rendering (such as some cloud environments or text editors), these color codes may not be appropriate as they can appear as raw text.

To accommodate for such scenarios, the `LoggingConfig.log_colors` option can be set to `false`. Disabling `log_colors` will prevent the addition of ANSI color codes to the log messages, which avoids raw escape codes in environments without color support.

Using a logger directly

It's possible to use `Logger` objects directly, and these can be initialized anywhere (very similar to the Python built-in `logging` API).

If you *aren't* using an object which already initializes a `NautilusKernel` (and logging) such as `BacktestEngine` or `TradingNode`, then you can activate logging in the following way:

```
from nautilus_trader.common.component import init_logging
from nautilus_trader.common.component import Logger

log_guard = init_logging()
logger = Logger("MyLogger")
```

See the `init_logging` API Reference for further details.

> ⚠ Only one logging subsystem can be initialized per process with an `init_logging` call. Multiple `LogGuard` instances (up to 255) can exist concurrently, and the logging thread will remain active until all guards are dropped.

LogGuard: managing log lifecycle

The `LogGuard` ensures that the logging subsystem remains active and operational throughout the lifecycle of a process. It prevents premature shutdown of the logging subsystem when running multiple engines in the same process.

Reference counting implementation

The logging system uses reference counting to track active `LogGuard` instances:

- **Counter increments**: When a new `LogGuard` is created, an atomic counter is incremented.
- **Counter decrements**: When a `LogGuard` is dropped, the counter is decremented.
- **Logging thread termination**: When the counter reaches zero (last `LogGuard` dropped), the logging thread is properly joined to ensure all pending log messages are written before the process terminates.
- **Maximum guards**: The system supports up to 255 concurrent `LogGuard` instances. Attempting to create more raises a `RuntimeError`.

This mechanism ensures that:

1. `LogGuard` keeps the logging thread alive and flushes on drop; abrupt termination (crashes, kill signals) can still lose buffered logs.
2. The logging thread remains active as long as any `LogGuard` exists.
3. On graceful shutdown, all buffered logs are properly flushed to their destinations.

Why use LogGuard?

Without a `LogGuard`, any attempt to run sequential engines in the same process may result in errors such as:

```
Error sending log event: [INFO] ...
```

This occurs because the logging subsystem's underlying channel and Rust `Logger` are closed when the first engine is disposed. As a result, subsequent engines lose access to the logging subsystem, leading to these errors.

By using a `LogGuard`, you can ensure consistent logging behavior across multiple backtests or engine runs in the same process. The `LogGuard` retains the resources of the logging

subsystem and ensures that logs continue to function correctly, even as engines are disposed and initialized.

> Using `LogGuard` is required to maintain consistent logging behavior throughout a process with multiple engines.

Running multiple engines

The following example demonstrates how to use a `LogGuard` when running multiple engines sequentially in the same process:

```
log_guard = None  # Initialize LogGuard reference

for i in range(number_of_backtests):
    engine = setup_engine(...)

    # Assign reference to LogGuard
    if log_guard is None:
        log_guard = engine.get_log_guard()

    # Add actors and execute the engine
    actors = setup_actors(...)
    engine.add_actors(actors)
    engine.run()
    engine.dispose()  # Dispose safely
```

Steps

- **Initialize LogGuard once**: The `LogGuard` is obtained from the first engine (`engine.get_log_guard()`) and is retained throughout the process. This ensures that the logging subsystem remains active.
- **Dispose engines safely**: Each engine is safely disposed of after its backtest completes. The `LogGuard` remains valid after `engine.dispose()` - only the engine is cleaned up, not the logging subsystem.
- **Reuse LogGuard**: The same `LogGuard` instance is reused for subsequent engines, preventing the logging subsystem from shutting down prematurely.

Considerations

- **Multiple LogGuards per process**: The system supports up to 255 concurrent `LogGuard` instances per process. Each guard increments a reference counter when created and decrements it when dropped.
- **Thread safety**: The logging subsystem, including `LogGuard`, is thread-safe, ensuring consistent behavior even in multi-threaded environments.
- **Automatic cleanup**: When the last `LogGuard` is dropped (reference count reaches zero), the logging thread is properly joined to ensure all pending logs are written before the process terminates.

Tracing subscriber for external Rust libraries

External Rust crates that use the `tracing` crate can have their log output displayed by enabling the tracing subscriber. This is useful for debugging external dependencies or when integrating custom Rust components (such as feature extractors or adapters) compiled as separate PyO3 extensions.

Enabling the subscriber

Enable the tracing subscriber by setting `use_tracing=True` in `LoggingConfig`:

```
from nautilus_trader.config import LoggingConfig
from nautilus_trader.config import TradingNodeConfig

config_node = TradingNodeConfig(
    trader_id="TESTER-001",
    logging=LoggingConfig(
        log_level="INFO",
        use_tracing=True,
    ),
    ... # Omitted
)
```

Alternatively, call `init_tracing()` directly:

```
from nautilus_trader.core import nautilus_pyo3

nautilus_pyo3.init_tracing()
```

Filtering with RUST_LOG

The `RUST_LOG` environment variable controls which tracing events are displayed:

```
# Show debug logs from your crate, warn and above from hyper
RUST_LOG=my_feature_extractor=debug,hyper=warn python my_script.py
```

If `RUST_LOG` is not set, the default filter level is `warn`.

How it works

The tracing subscriber uses a `tracing-subscriber` fmt layer with a custom formatter to output directly to stdout. This is separate from the Nautilus logging infrastructure - tracing output uses a Nautilus-aligned format with nanosecond timestamps.

Example tracing output:

```
2026-01-24T05:51:42.809619000Z [DEBUG] hyper_util::client::legacy::connect::http: connecting to 1
2026-01-24T05:51:42.810543000Z [DEBUG] hyper_util::client::legacy::pool: pooling idle connection
```

Differences from Nautilus logging:

- Tracing output goes directly to stdout, not through the Nautilus logging thread.
- Tracing events are not written to Nautilus log files.
- Filtering is controlled exclusively by `RUST_LOG`, independent of `LoggingConfig`.

For external libraries that use the `log` crate (such as `rustls`), their events go through the Nautilus logger and are filtered by `log_level` / `log_level_file` in `LoggingConfig`.

> `RUST_LOG` only affects crates using `tracing`. For crates using `log`, configure verbosity via `LoggingConfig` or the `NAUTILUS_LOG` environment variable (e.g., `NAUTILUS_LOG=stdout=Debug`).

> The tracing subscriber can only be initialized once per process. When using `use_tracing=True` in `LoggingConfig`, subsequent kernel creations safely skip re-initialization. Direct calls to `init_tracing()` when already initialized will raise an error.

Platform-specific considerations

Windows shutdown behavior

On Windows, non-deterministic garbage collection during interpreter shutdown can occasionally prevent the logging thread from joining properly. When the last `LogGuard` is dropped, the logging subsystem signals the background thread to close and joins it to ensure all pending messages are written. If Python's garbage collector delays dropping the guard until after interpreter shutdown has begun, this join may not complete, resulting in truncated logs.

This issue is tracked in GitHub issue #3027. A more deterministic shutdown mechanism is under consideration.

Related guides

- Architecture - System architecture including logging infrastructure.

‹ Reports
Previous Page

Backtesting ›
Next Page